

SurveySmart — Code Review Fixes

Full backend + frontend review conducted, all identified issues fixed and verified. Project: D:\Claude_workspace\surveysmart · Date: 2026-07-20

Fixes Applied

1. Google OAuth state not scoped to a user

SECURITY

Files: backend/src/services/googleAuth.js, backend/src/routes/googleForms.js

Why it mattered: The OAuth “state” token used to link a Google authorization popup back to a SurveySmart action (create form / import form / sync responses) was stored in a plain in-memory Set with no owner. Any authenticated user who obtained another user's in-flight state value could use it to access that user's Google tokens for the 10-minute window it was valid.

How it was fixed: Added a `stateOwners` map that records which user's request registered each state. `registerPendingState(state, userId)` now takes the user id, and `getTokens(state, userId)` only returns tokens if the state was registered by that same user. All five call sites in `googleForms.js` (`auth-url`, `create-form`, `import-auth-url`, `import-form`, `sync-auth-url`, `sync-responses`) were updated to pass `req.user.id`.

2. Falsy-zero bug in scale question min/max

BUG

Files: backend/src/controllers/responseController.js, frontend/src/views/SurveyFillView.vue

Why it mattered: Both the `chart-data` endpoint and the public fill form read a question's configured minimum with `opts?.min || 1`. Because `0` is falsy in JavaScript, a survey author who set a scale's minimum to `0` would silently have it overridden to `1` everywhere the value was read.

How it was fixed: Replaced `||` with the nullish-coalescing operator `??` in both places, so only `null/undefined` falls back to the default — an explicit `0` is now respected.

3. Google Forms webhook endpoint had no request authentication

SECURITY

Files: backend/src/routes/publicResponses.js, backend/.env.example

Why it mattered: `POST /api/responses/public/form/:formId` accepts response submissions with no auth beyond a per-IP rate limiter. Anyone who learned a survey's `google_form_id` (not treated as secret elsewhere) could forge arbitrary response data into the database.

How it was fixed: Added an optional shared-secret check (`verifyWebhookSecret` middleware). If the new `GOOGLE_FORMS_WEBHOOK_SECRET` environment variable is set, the request must include a matching `X-Webhook-Secret` header or `?secret=` query parameter, or it's rejected with `401`. If the variable is left unset, the check is skipped so existing deployments keep working unchanged — documented in `.env.example`, with a note recommending it be set in production (the Apps Script webhook script must then be updated to send the

secret).

4. Stray duplicate file left in the repo

CLEANUP Files: frontend/src/components/Survey/SurveyBuilder.vue.testcopy (deleted)

Why it mattered: A byte-for-byte duplicate of SurveyBuilder.vue with a .testcopy extension was sitting in the components folder, unused by any import — dead clutter that could confuse future edits (e.g. someone editing the wrong copy).

How it was fixed: Deleted the file after confirming (via diff) it was identical to the real component and not referenced anywhere in the codebase.

5. Status-badge / score-interpretation logic duplicated across 5 files

**REFACTO
R** Files: New: frontend/src/composables/useSurveyStatus.js | Updated: SurveysView.vue, SharedView.vue, DashboardView.vue, ResponsesView.vue, SurveyCard.vue

Why it mattered: formatDate, badgeClass, badgeText, interpClass, and interpText were copy-pasted verbatim into multiple views. Any future change to badge colors or score-interpretation thresholds would need to be made consistently in up to five places by hand — a drift risk.

How it was fixed: Extracted the identical logic into one composable, useSurveyStatus.js, and updated each view to import from it instead of redefining locally. ResponsesView.vue keeps its own formatDate (it intentionally uses a different date format/locale for that table) but now shares the badge/interpretation helpers with everyone else.

Verification

- ✓ **Frontend:** vite build completed cleanly (184 modules, no errors); the new useSurveyStatus.js composable was correctly split into its own chunk in the production bundle.
- ✓ **Backend:** node --check passed on every edited file (googleAuth.js, googleForms.js, responseController.js, publicResponses.js, app.js) — no syntax errors.
- ✓ **Grep check:** confirmed no leftover duplicate local function definitions after the composable refactor, and that all five Google OAuth call sites were updated consistently.

Reviewed but intentionally not changed

A few items surfaced during the review that are architectural trade-offs rather than bugs — changing them would be a larger design decision, so they were left as-is and are flagged here for awareness:

- **JWT stored in localStorage** (frontend/src/stores/auth.js, api/index.js) — standard for SPAs but vulnerable to token theft via XSS; the httpOnly-cookie alternative would require a broader session-handling rewrite on the backend.

- **Google OAuth token store is in-memory** (`services/googleAuth.js`) — tokens/state are lost on server restart and won't work correctly across multiple backend instances. Fine for a single-instance deployment; would need Redis or similar to scale horizontally.
- **`adminController.setRole` only allows 'user'/'admin'** — there's no path to promote/demote a `head_admin` through this endpoint. Left unchanged since it's unclear whether that's a deliberate restriction; worth confirming the intended behavior before changing it.